MMLC Game Creator 1 Plugin Documentation

None

Threepeat Games

Threepeat, LLC

Table of contents

1. Overview

This plugin for MMLC provides automatic support for integrating MMLC into Game Creator characters.

> **Important**
>
> **Please read this page and This other page prior to purchase/use!**

> **Important**
>
> **This asset requires Motion-Matching Locomotion Controller (MMLC) and Game Creator 1 (GC1) and can't work without it.** Also note that MMLC itself has dependencies on MxM and Movement Animset Pro for proper function (although the Movement Animset Pro requirement can be waived if you don't require crouch modes).

> **Important**
>
> This asset currently only supports Game Creator 1, but **Game Creator 2 (GC2) support is coming soon** and GC2 support will be added to this asset as an update at no additional cost.

1.1 Included Features:

This plugin integrate all of our Motion-Matching Controllers with Game Creator. It includes everything you need to use MMLC locomotion and MMPC parkour with Game Creator 1. **Game Creator 2 support coming soon**

This asset contains the following for Game Creator (GC) 1:

- Automatic configuration and setup of all of our Motion-Matching controllers with GC via our Config Wizard, which also includes optional one-click integration with Final IK and Strider.
- All actions/instructions, triggers, and conditions needed to switch between our controllers and GC or use both simultaneously with Avatar Masks
- Examples demonstrating use with GC core, melee, shooter, and traversal modules

> **Note**
>
> Game Creator 2 support will be added to this asset in a future update.

This asset is in active development, with more features to be added periodically based both on planned additions and user feedback (see Discord link below to give feedback)!

For support and requests, use our Discord or email threepeatgames@gmail.com.

2. Installation

2.1 Supported Unity/Pipeline Version

This asset works in 2019+, but 2021 LTS or later is recommended.

2.2 Definitions and Notes

- **GC1** - Game Creator 1 by Catsoft Works
- **MMLC** - Motion-Matching Locomotion Controller (This asset)
- **MxM** - Motion-Matching for Unity (Animation Uprising) asset
- **MAP** - Movement Animset Pro (Kubold) asset

Philosophy for MMLC and GC1 Installation/Use

With MMLC and the GC1 plugin asset, it's important to think in terms of two major steps to do any functionality addition and testing:

```
1. Perform MMLC steps and confirm function on regular MMLC characters.
1. Perform any Follow-on GC1 steps and confirm function on GC1 characters.
```

For example, to get a new animation set imported and working with GC1, you would first follow the installation steps in the MMLC documentation here and then perform the GC1-specific installation steps at the bottom of this documentation section.

2.3 Installations steps 1 - Creating Project and Installing Dependencies

Follow the MMLC installation instructions here: MMLC Installation

2.4 Installation steps - 2 - Installing GC1 and GC1 Plugin for MMLC

Note

These instructions are for Unity 2021 LTS. If any of these steps don't work exactly as expected in your Unity version, google likely has you covered, but please reach out on discord if you're stuck.

If you prefer a video walkthrough of these steps, that can be found here:

1. Import `Game Creator` (GC1) from the Package Manager. Consult Game Creator 1 documentation and Catsoft's youtube channel for assistance with this step. Don't forget to click `Install Game Creator 1.1.16` button in the popup that appears.

2.5 Installation steps - 3

1. Open the relevant example scene for your configuration to confirm all advertised functions work as expected before moving on to setting up custom characters and behavior. The table below shows appropriate example scene based on installed dependencies (MxM and MAP are required and assumed to be installed):

   | Purpose | Demo Scene (in Assets/Plugins/Threepeat/MMCGameCreator) | | -------- | ---------- | ---- | | Core MMLC/GC1 Functionality | Demos/ExampleScene | | Melee | | | Shooter | | | Traversal | |

   Information on key bindings and features of the demo scenes can be found here: Demo Scene

   A video walkthrough of Testing the Installation can be found here:

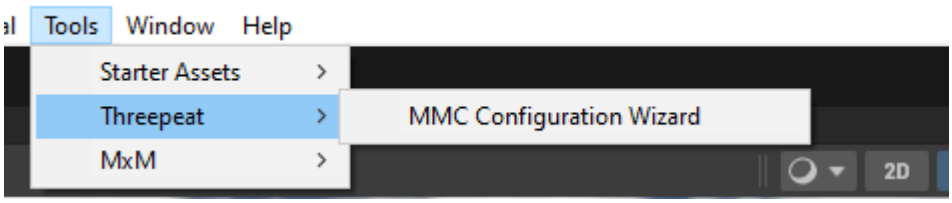
3. Configuration - Characters

3.1 Configuration Wizard

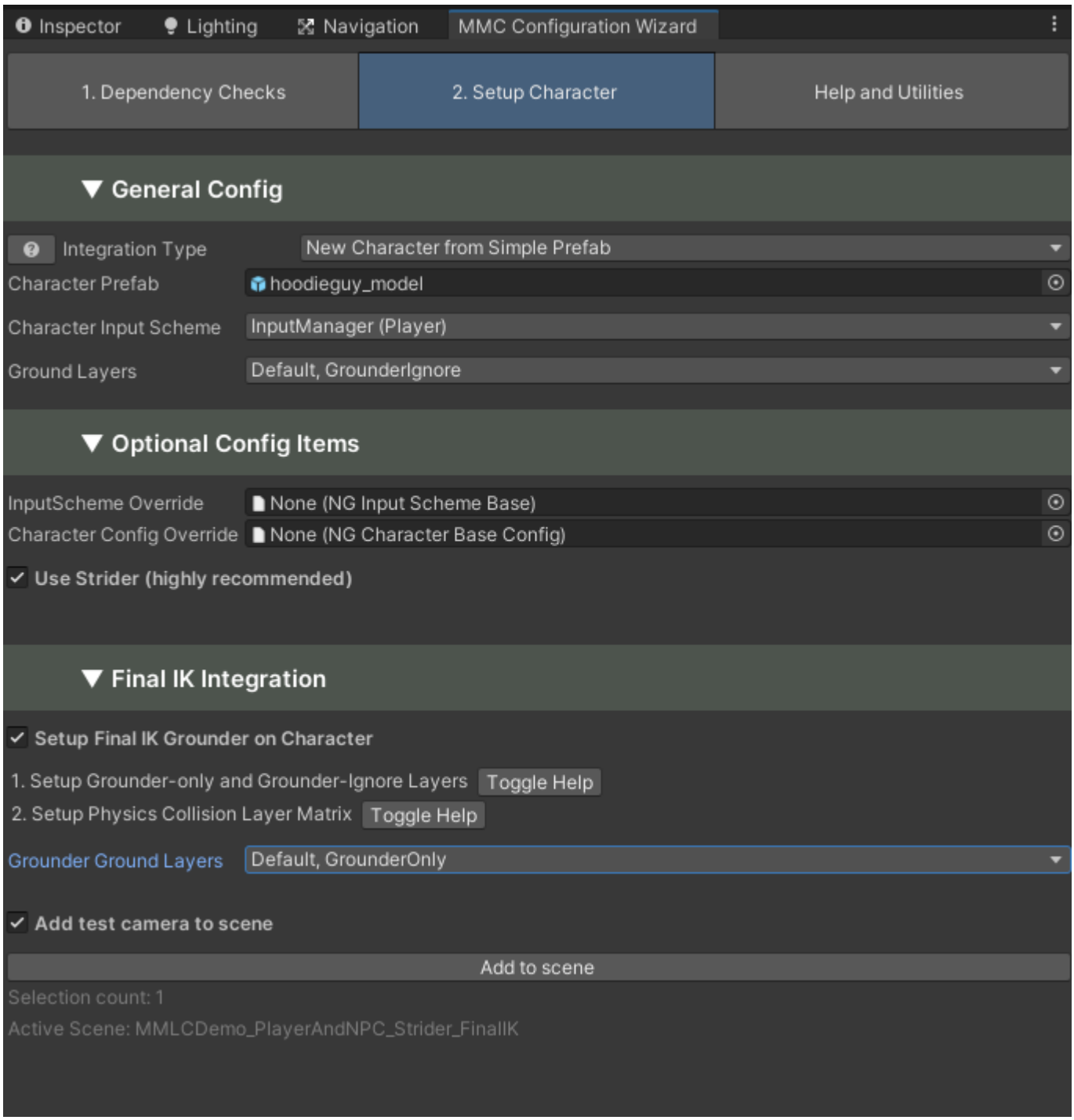
This asset comes with a configuration wizard that attempts to:

- Make player and Non-Player Character (NPC) setup automatic
- Automatically check and validate all dependency packages and integrations are installed in the project
- Assist with correcting any MMLC-related configuration issues in the project

To access the wizard, go to **Tools->Threepeat->MMC Configuration Wizard**



The wizard window should appear, as shown below (In image, the window has been docked with the Inspector):



The wizard has three main tabs: "Dependency Checks", "Setup Character", and "Help and Utilities"

Always start with Dependency Checks.

3.1.1 **Dependency Checks**

3.1.2 **Help and Utilities**

This tab contains a button to take you to the latest version of this help, which is located here.

Additionally, this tab contains helper utilities. The only currently-exposed utility is "Add Footstep Events to Movement Animset Pro", which will insert all necessary **NGFootstepEvent** and **NGRawAudioEvent** Animation Events to make the necessary events in the **NGDispatchAnimationEvents** component fire.

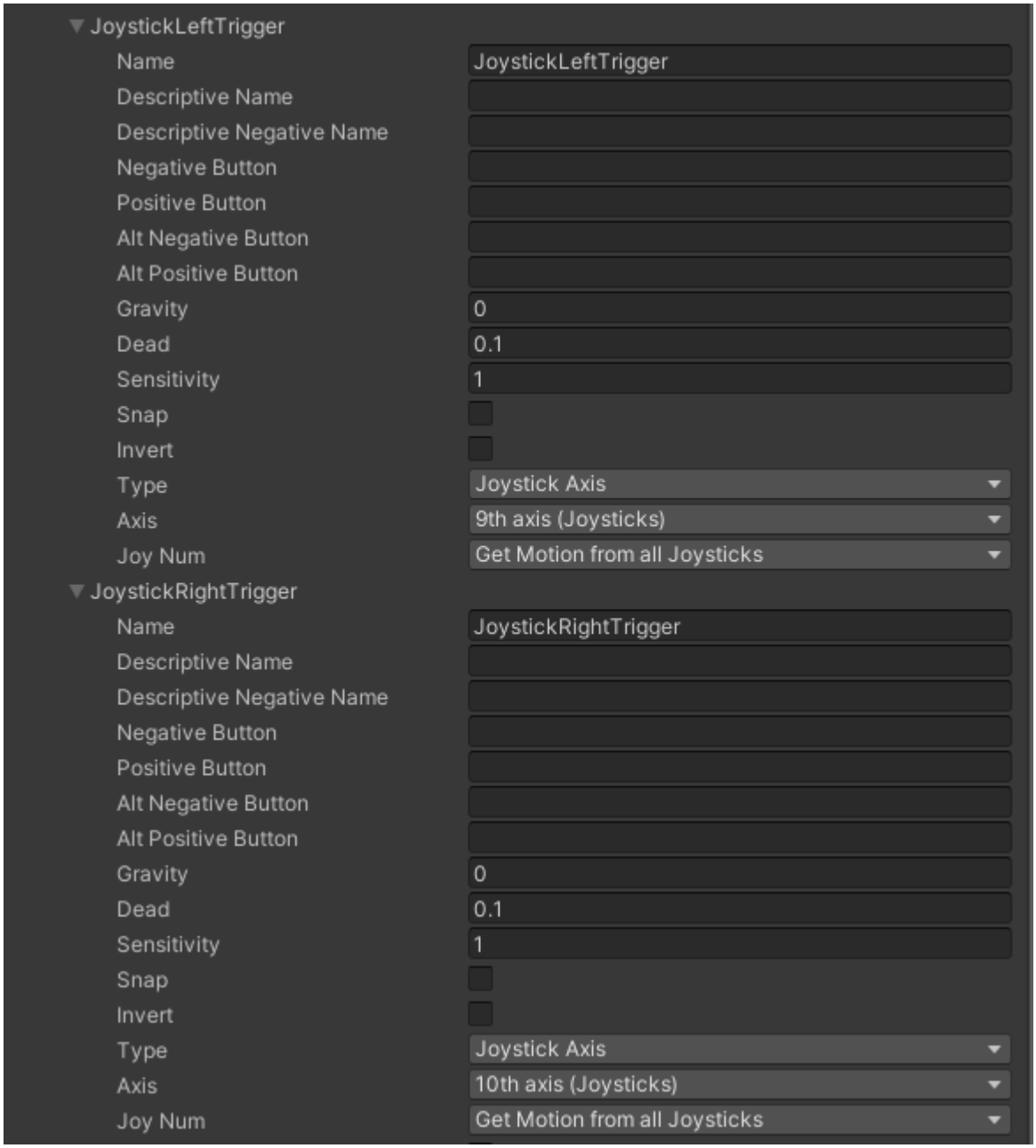
This will allow you to add audio, visual effects, foot prints, and any other kind of reactions needed in response to foot steps. See Audio Events for more information

3.2 Setting up an input-driven character (Player)

The new InputSystem is highly recommended, but legacy InputManager is also supported by MMLC and the configuration wizard.

3.2.1 Joystick Support (InputManager)

1. The joystick left trigger is used for sprint, and should be named JoystickLeftTrigger and configured in **Project Settings -> Input Manager** as shown in the below picture:



1. In the PlayerCharacter's root object (the object with NGPlayerController component), set **use Joystick Axes** to true.

3.3 Setting up a Non-Player character (AI/NPC)

In addition to standard NavMeshAgent::SetDestination usage, NGNonPlayerCharacter offers a more persistent target transform that allows MMLC to manage reaction to target changes. This can be set/cleared as shown in **Assets/Plugins/Threepeat/MMCLocomotion/Scripts/Examples/Example_SetNavMeshTargetOnLKey**.

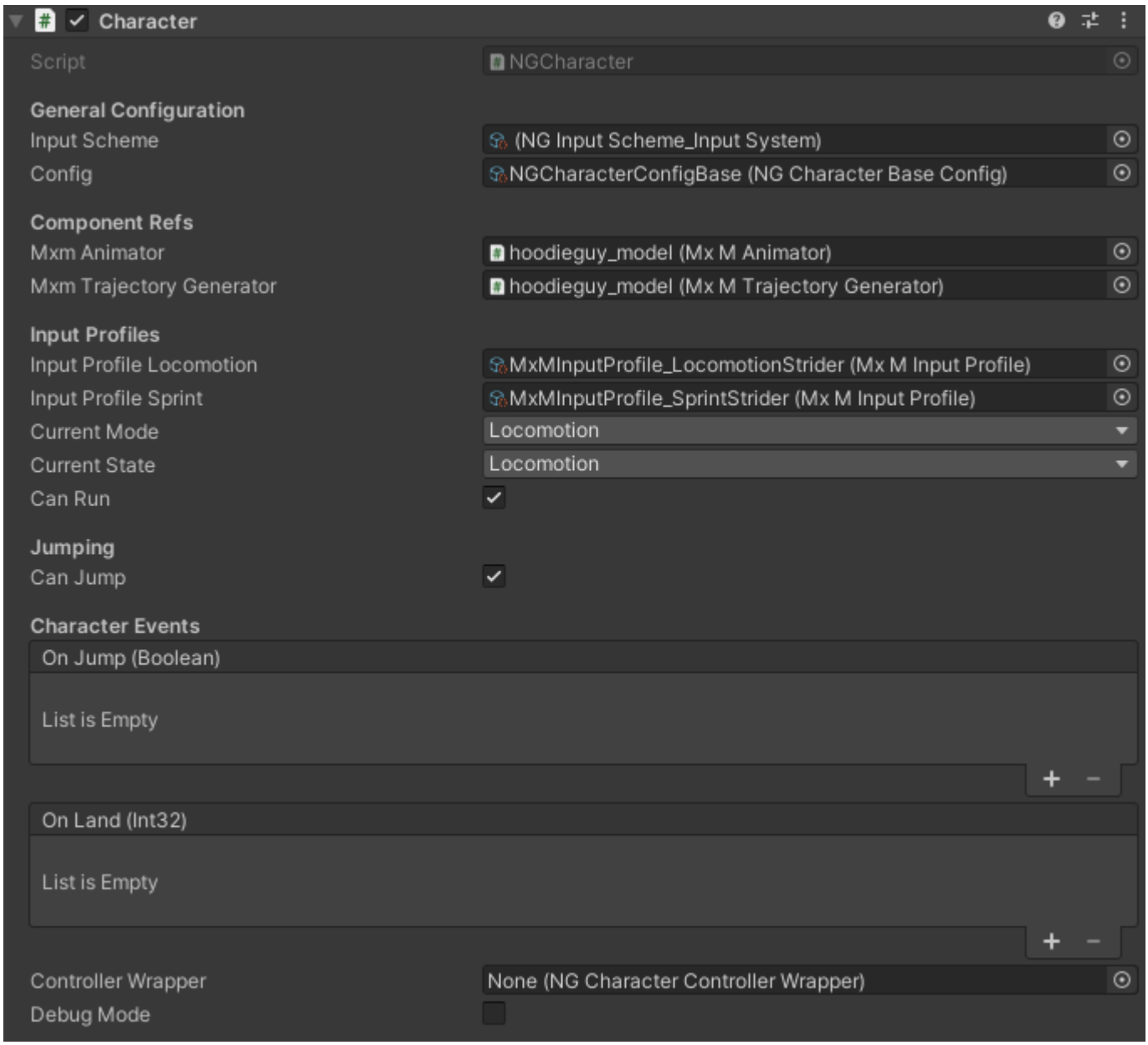
To change/clear a target for any NavMesh-driven character (code assumes it's running in a MonoBehaviour on the same GameObject as the NGCharacter component):

```
character = GetComponent<NGCharacter>();
NGInputScheme_NavMesh inputScheme = (NGInputScheme_NavMesh)character.InputScheme;
inputScheme.currentTarget = yourNewTargetTransform; // or null if you wish to clear the target.
```

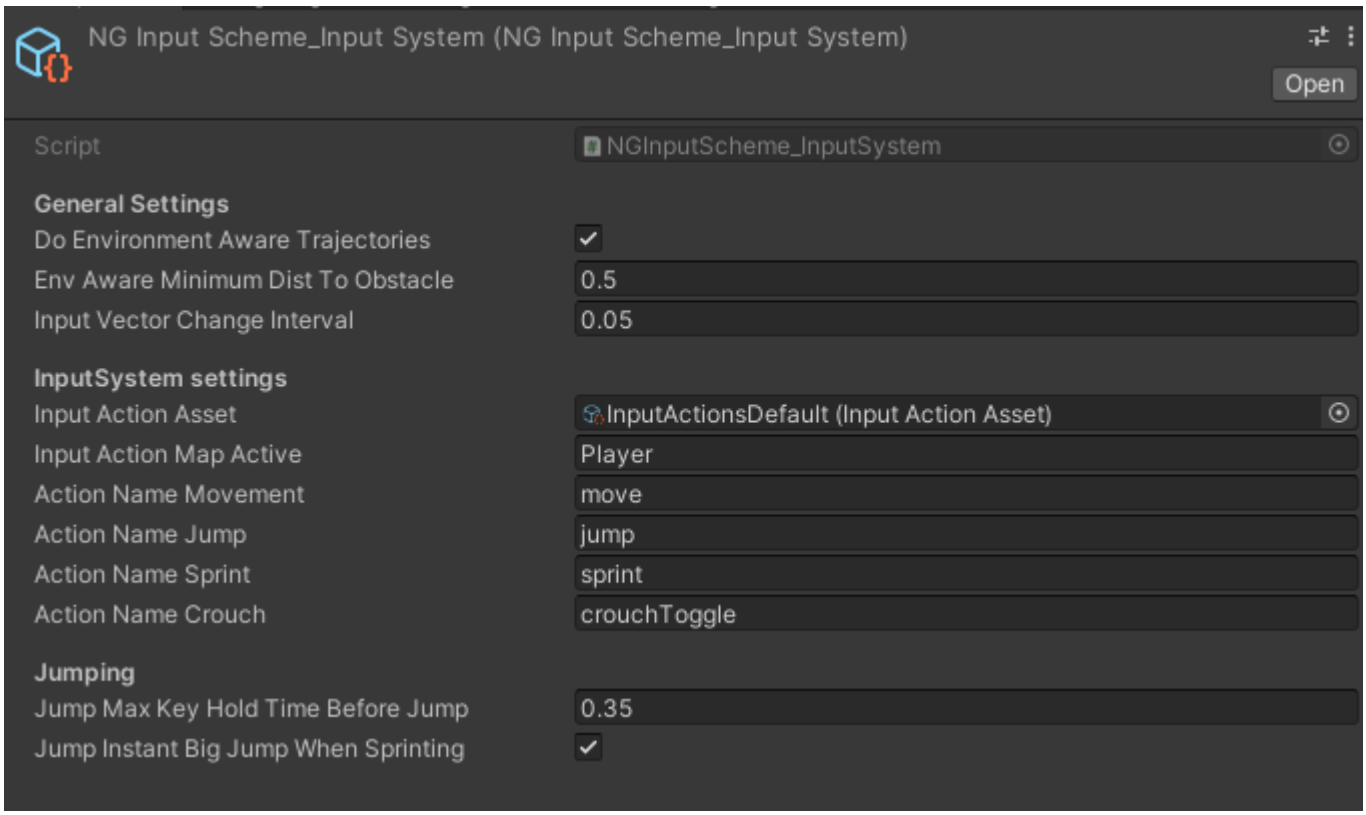
3.4 Tuning Character Behavior for Players and NPC's

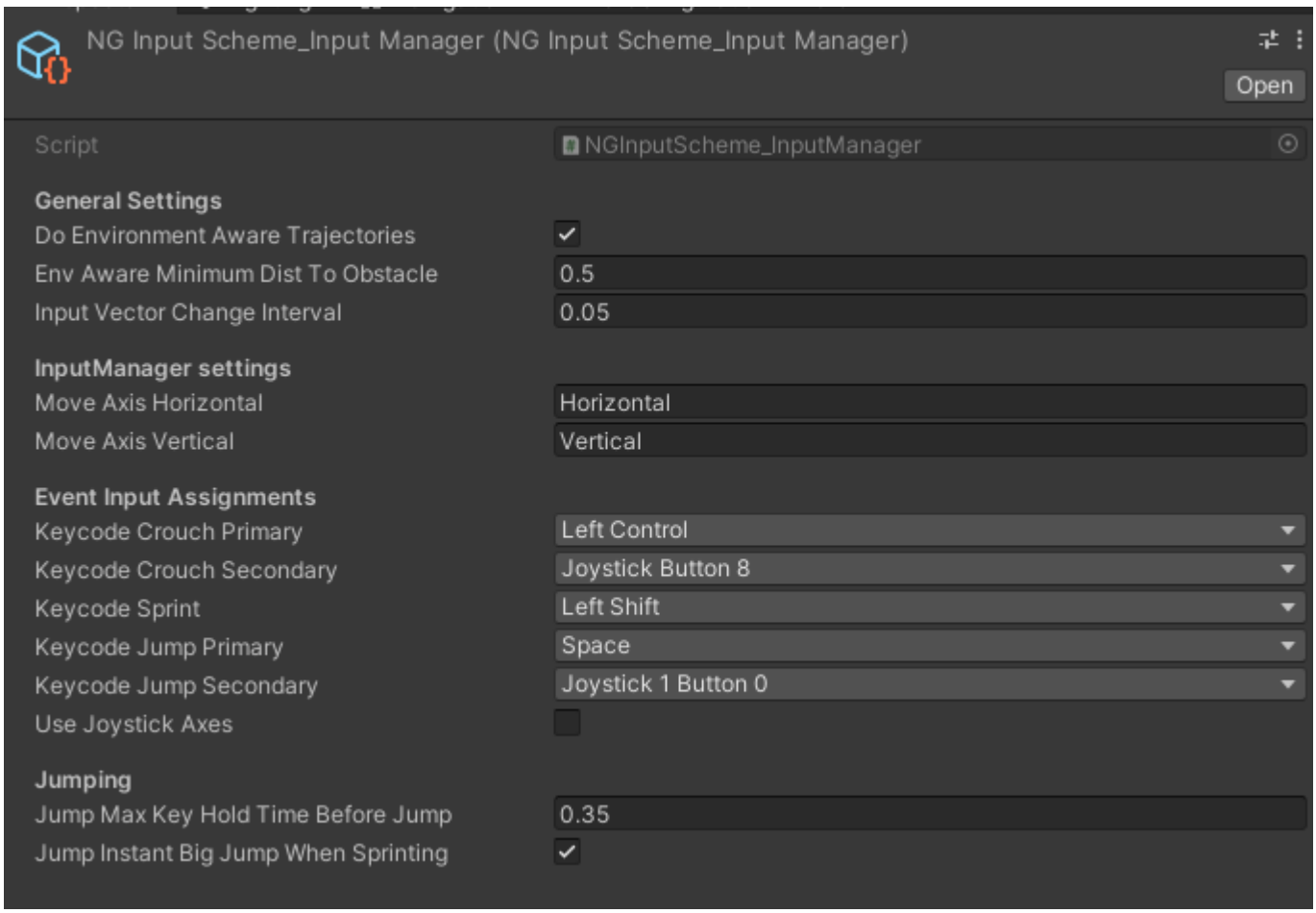
3.4.1 Player Only - Controls/Input

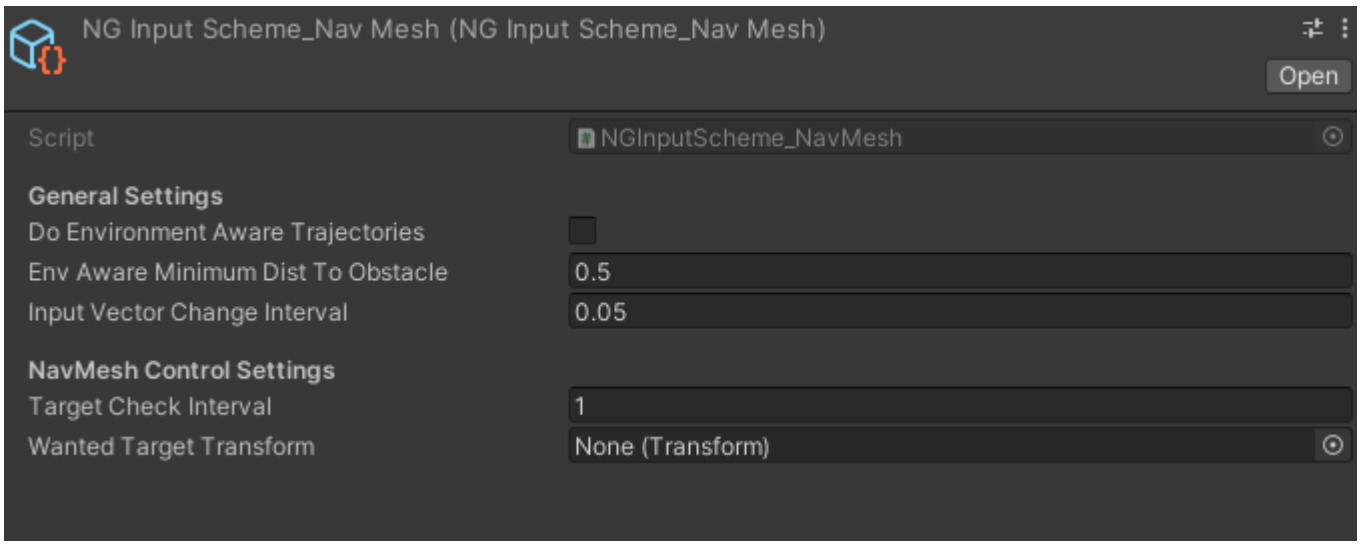
The NGCharacter component (and inspector) has an Input Scheme property that points to a Scriptable Object (see "Input Scheme" in image below) that controls all aspects of what drives a character to perform input-related actions. There are schemes for InputSystem, InputManager, and NavMesh (NPC driven by a target on the navmesh).



The inspectors for each of the three InputScheme types are below:


NG Input Scheme_Input System (NG Input Scheme_Input System)
Open
Script
NGInputScheme_InputSystem
General Settings
Do Environment Aware Trajectories
Env Aware Minimum Dist To Obstacle
0.5
Input Vector Change Interval
0.05
InputSystem settings
Input Action Asset
InputActionsDefault (Input Action Asset)
Input Action Map Active
Player
Action Name Movement
move
Action Name Jump
jump
Action Name Sprint
sprint
Action Name Crouch
crouchToggle
Jumping
Jump Max Key Hold Time Before Jump
0.35
Jump Instant Big Jump When Sprinting


NG Input Scheme_Input Manager (NG Input Scheme_Input Manager)
Open
Script
NGInputScheme_InputManager
General Settings
Do Environment Aware Trajectories
Env Aware Minimum Dist To Obstacle
0.5
Input Vector Change Interval
0.05
InputManager settings
Move Axis Horizontal
Horizontal
Move Axis Vertical
Vertical
Event Input Assignments
Keycode Crouch Primary
Left Control
Keycode Crouch Secondary
Joystick Button 8
Keycode Sprint
Left Shift
Keycode Jump Primary
Space
Keycode Jump Secondary
Joystick 1 Button 0
Use Joystick Axes
Jumping
Jump Max Key Hold Time Before Jump
0.35
Jump Instant Big Jump When Sprinting



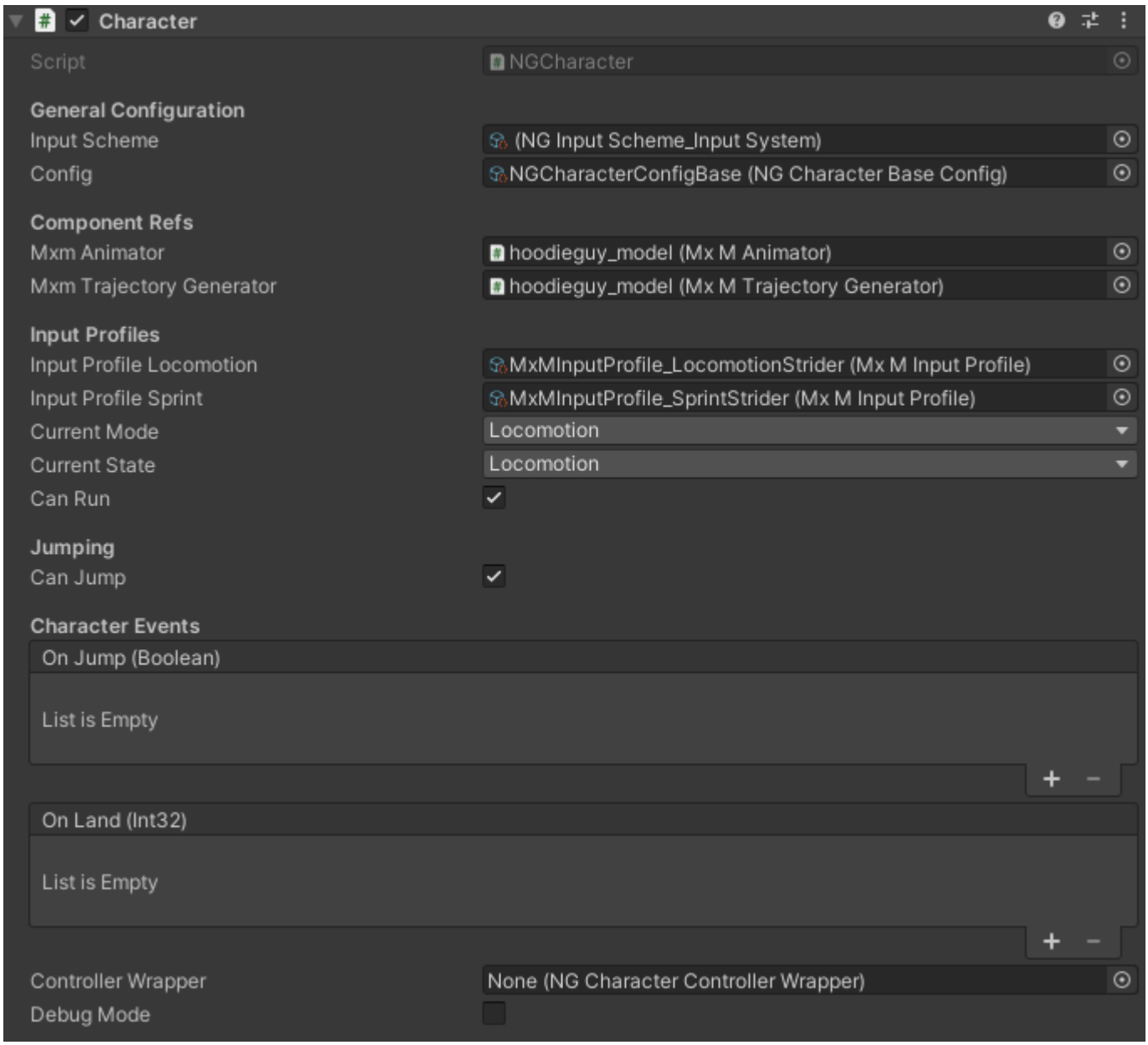
Important

Unlike the config property, a fresh input scheme object is instantiated (cloned) for each character, as otherwise, e.g. all common NavMesh-controlled agents would only be able to move to the same destination.

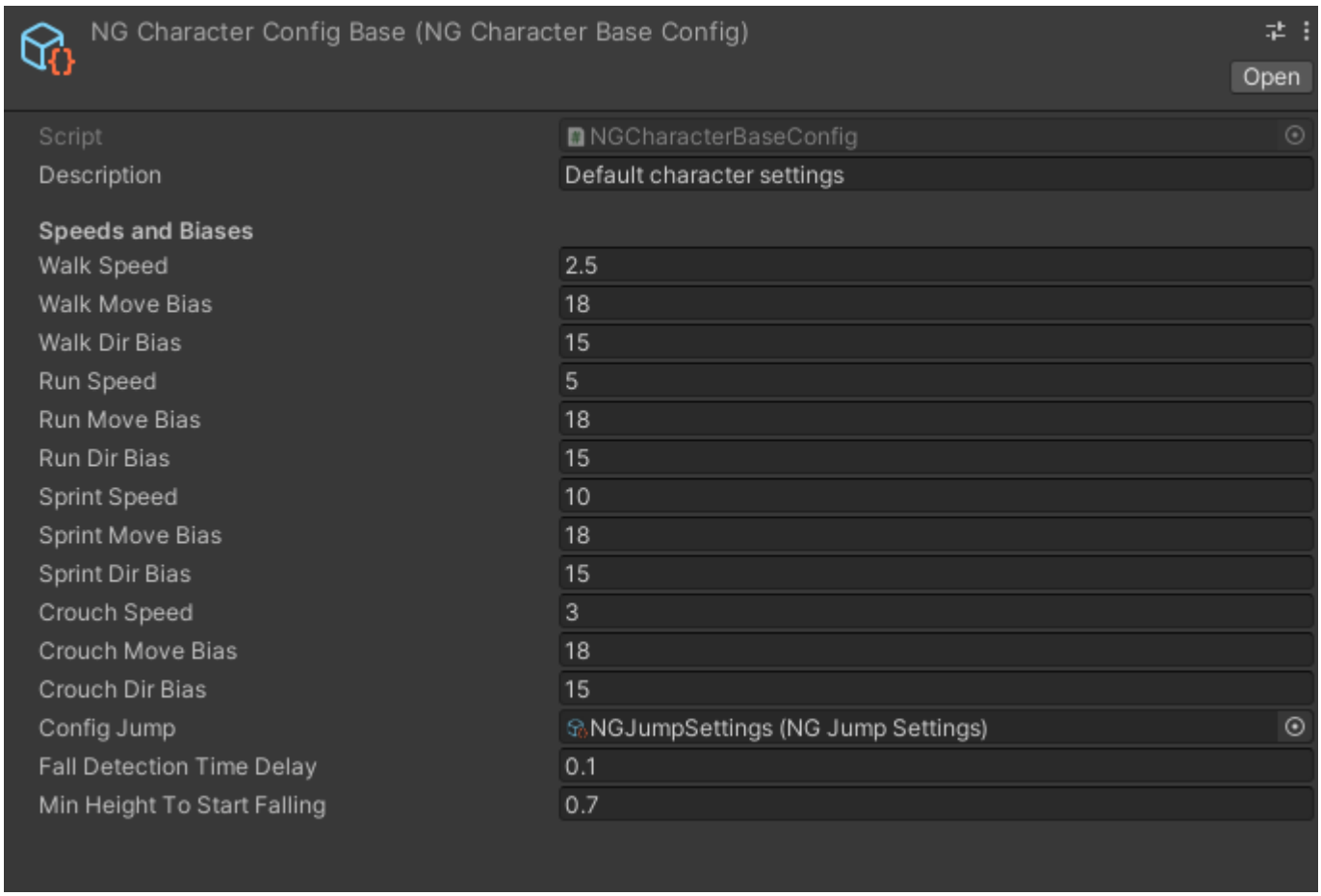
In the *Jumping** section of the InputSystem and InputManager inspectors, shown above, how long the jump key must be held down before a big jump is executed can be configured (set to 0 or a very small number of seconds (e.g. 0.001) for instant big jump), as well as whether to perform an instant big jump when the jump key is pressed while sprinting. In all other cases, the jump will occur on key up or after the max key hold time before jump has elapsed since the key was pressed.

3.4.2 Common - Run/Sprint/Crouch-walk speed, Jump enable/disable, Falling configuration

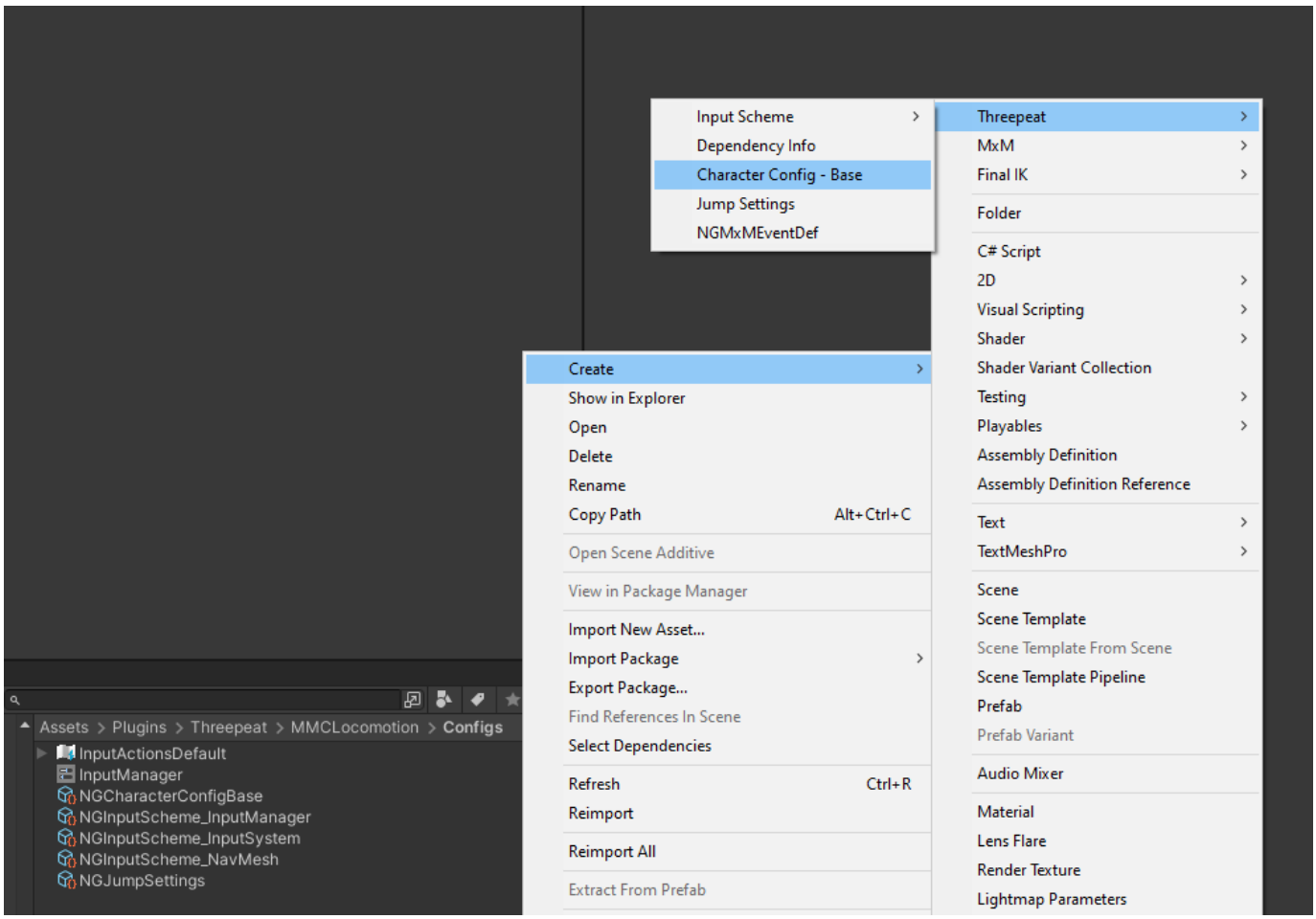
The NGCharacter component (and inspector) has a config property (see "Config" in image below) that points to a Scriptable Object that controls the character's locomotion and jumping behavior.



Here is the inspector for that Scriptable Object:



These can be found in **Assets/Plugins/Threepeat/MMCLocomotion/Configs** and created via right-click menu as shown below:



To adjust walk and run speeds of a character, change **Walk and Run Speed** in the **Speeds and Biases** section of the NGCharacter config scriptable object. For sprint, adjust **Sprint Speed**, and crouch-walk speed can be adjusted via **Crouch Speed**.

While jumping is predominantly configured via the reference NGJumpSettings ScriptableObject (discussed in next section), the **Jumping** section of the controller contains the toggle for whether a character is able to jump at all.

For falling detection, **Fall Detection Time Delay** configured the time after the character is no longer grounded (there is no longer any ground surface directly under their feet) before the character goes into a "Falling" state (with accompanying fall- and forward- speed-based animations). **Min Height To Start Falling** defines the minimum distance between the character's feet and the ground for falling to be engaged.

> **Note**
>
> Since v0.2.0 on, All common settings (applicable to both Player and NPC characters) have been rolled into a top-level NGCharacterConfigBase ScriptableObject to make character classes/types more portable, easier to define, and easier to drop into characters at scale. This also allows wholesale changing of behavior for all commonly-configured characters with a single object modification.

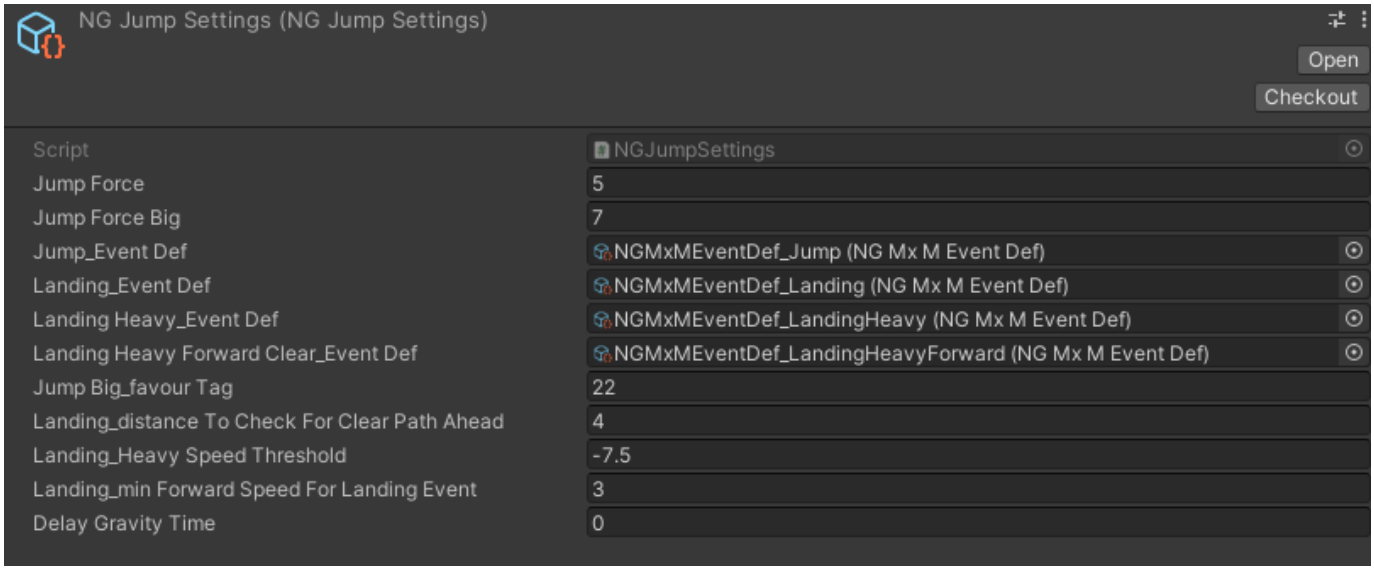
The inspector screenshot above also highlights the two major events available in the core character class:

- On Jump (boolean: true if big jump, false otherwise)
- On Land (int: int value of NGCharacterBase.LandingType enum which defines the types of landing available in the controller)

3.4.3 Common - Deeper Jumping Configuration

The below NGJumpSettings ScriptableObject allows different jump behavior to be configured. The items can be configured:

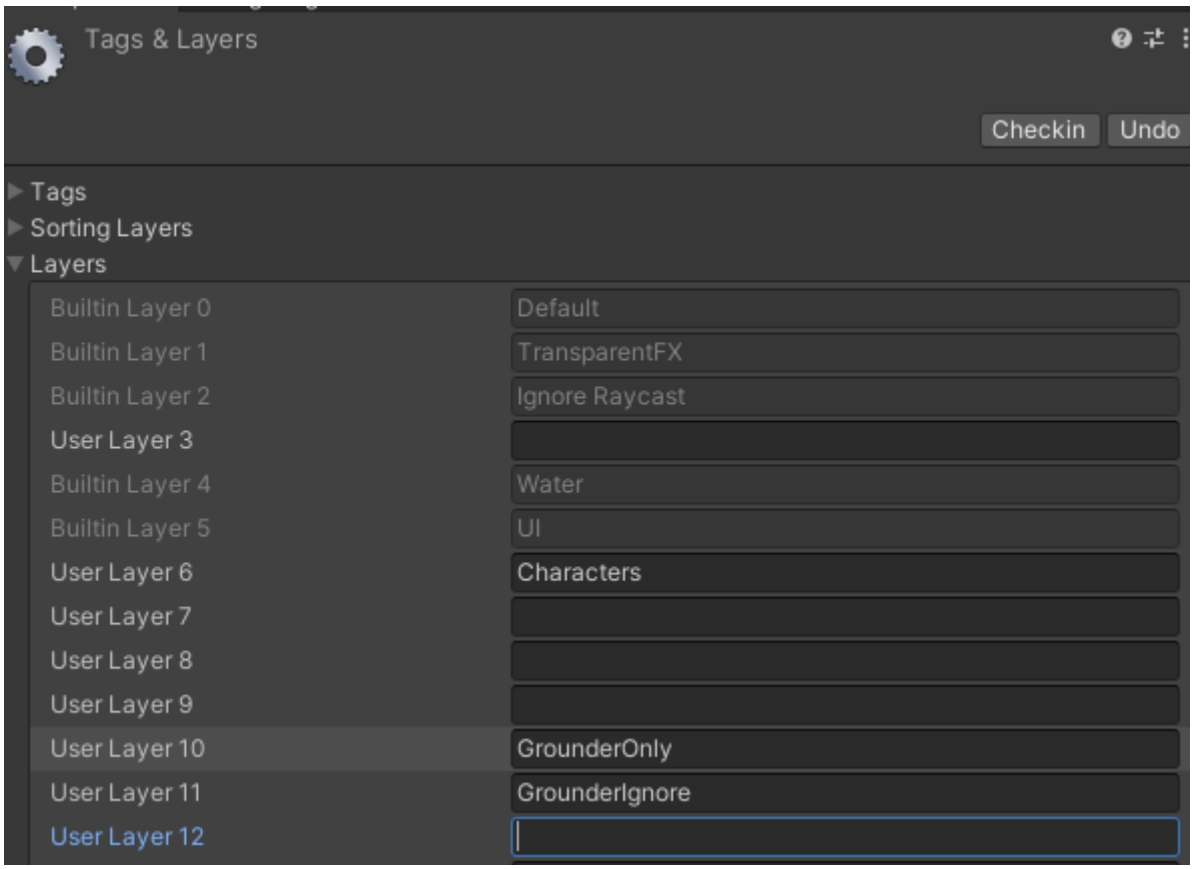
- **Jump Force**: upward force to apply for regular jumps
- **Jump Force Big**: upward force to apply for big jumps
- **Landing_Distance to Check for Clear Path Ahead**: Distance ahead of character at landing to check for obstructions. If no obstructions found, the player will perform a landing roll. Otherwise, the player will perform a stationary heavy landing.
- **Landing_Heavy Speed Threshold**: Minimum downward speed (ground impact speed) to trigger a "Heavy" landing.
- **min Forward Speed For Landing Event**: For normal landings below the heavy speed threshold, this defines the minimum forward speed to engage moving landing animations.



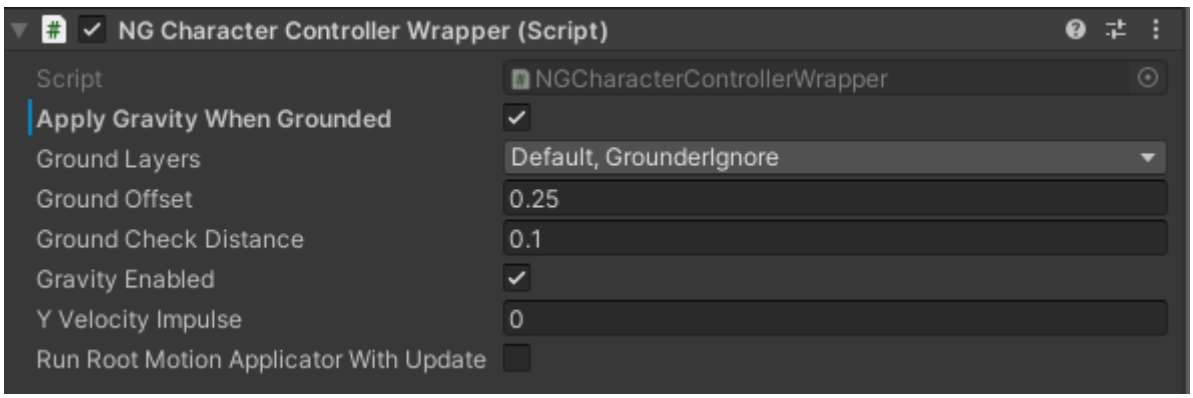
3.5 Setting up your Character and Environment for Final IK grounder

3.5.1 **Environment**

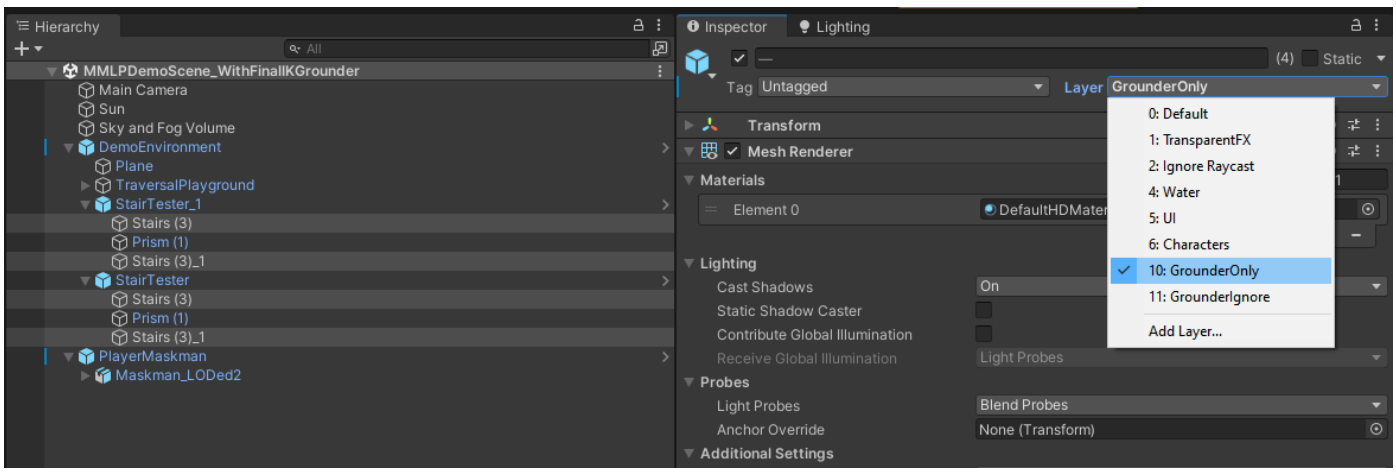
1. Create Layers: GrounderOnly and GrounderIgnore



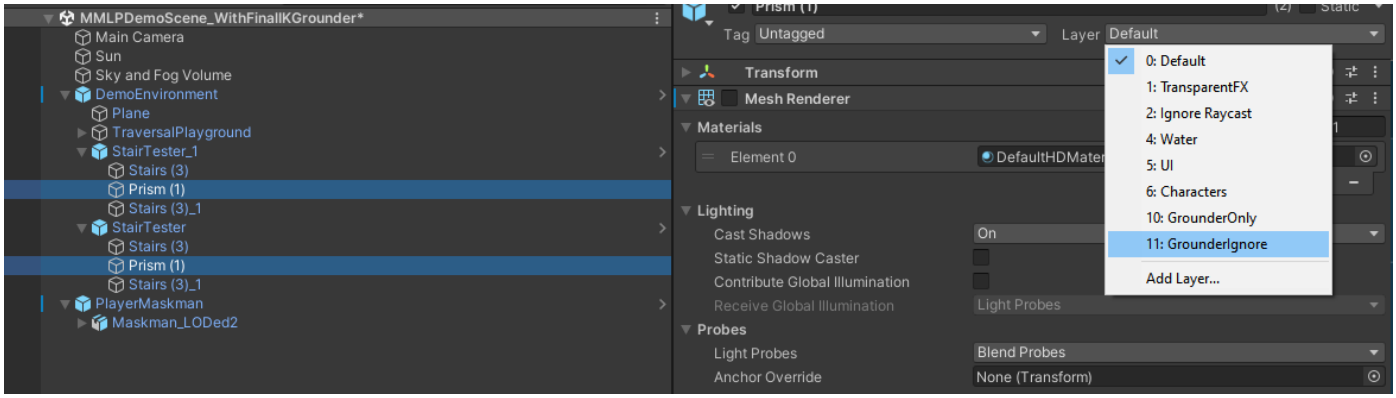
2. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)



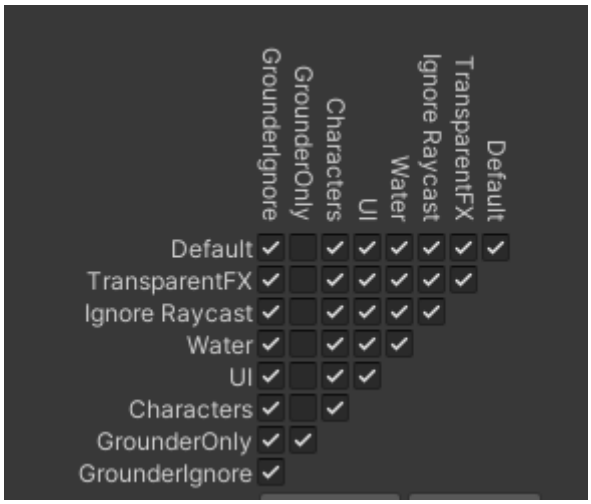
3. Put all stepped-incline objects (stairs) on **Grounder Only** layer



4. Put put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



5. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:

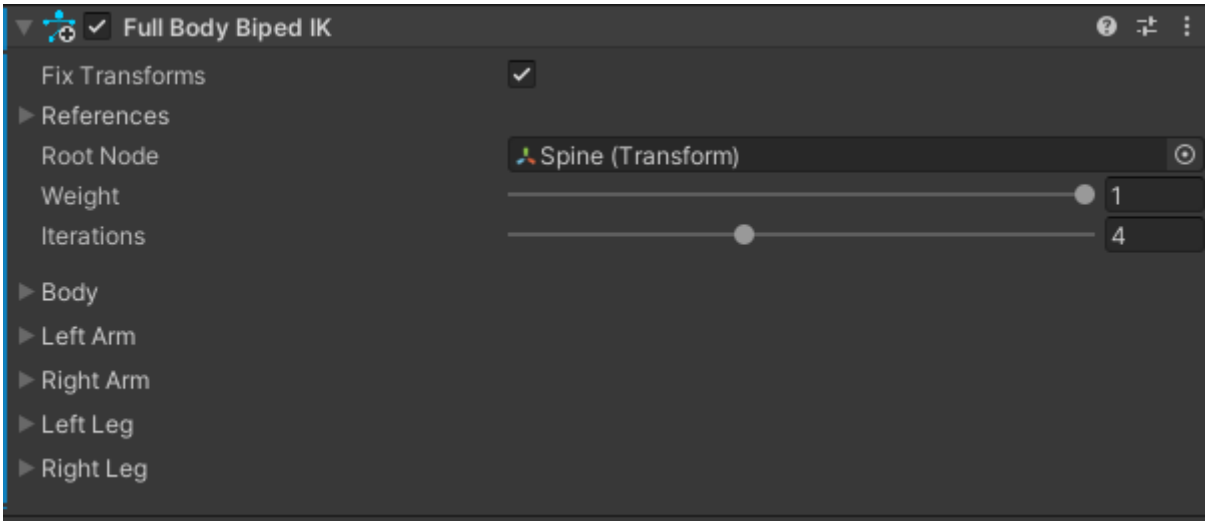


3.5.2 **Character**

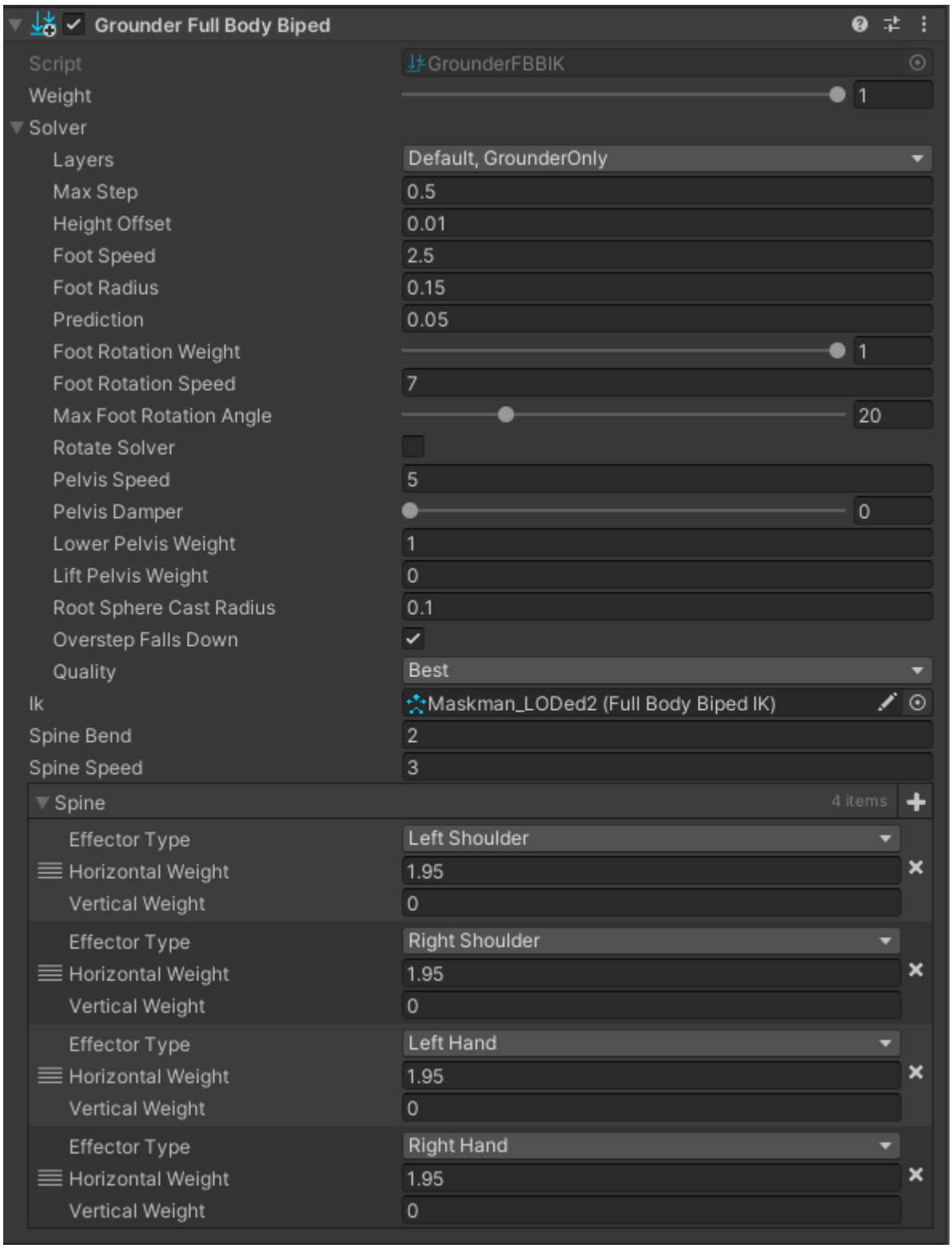
Important

This section is provided for general reference. If you're using our Motion-Matching Locomotion Controller, the config wizard will automatically configure The Biped IK and Grounder components for you. However, if you wish to perform this step manually, or if you don't any Threepeat assets and just want to set up grounder, here you go:

1. To the sub-object of your character that contains the Animator (and MxMAnimator, etc), Add **FBBIK** Component:



2. To that same object, add **Grounder Full Body Biped** component and set settings per the picture below:



a. Make sure All ground layers (except Grounder Ignore) are set in Solver -> Layers

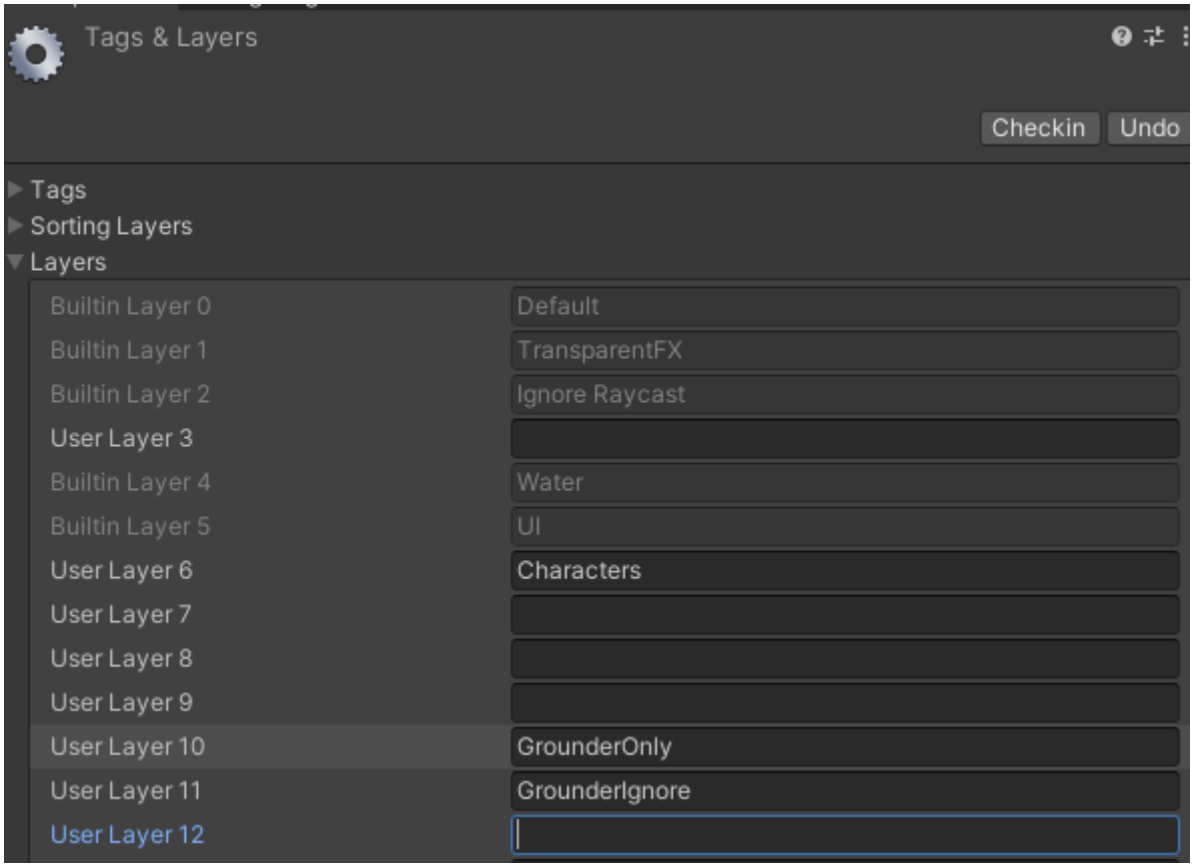
b. Add the Left/Right Shoulders and Left/Right Hands to the Spine list (this causes the player to lean forward when running up inclines and stay more upright when descending inclines to add realism)

3. You're done!

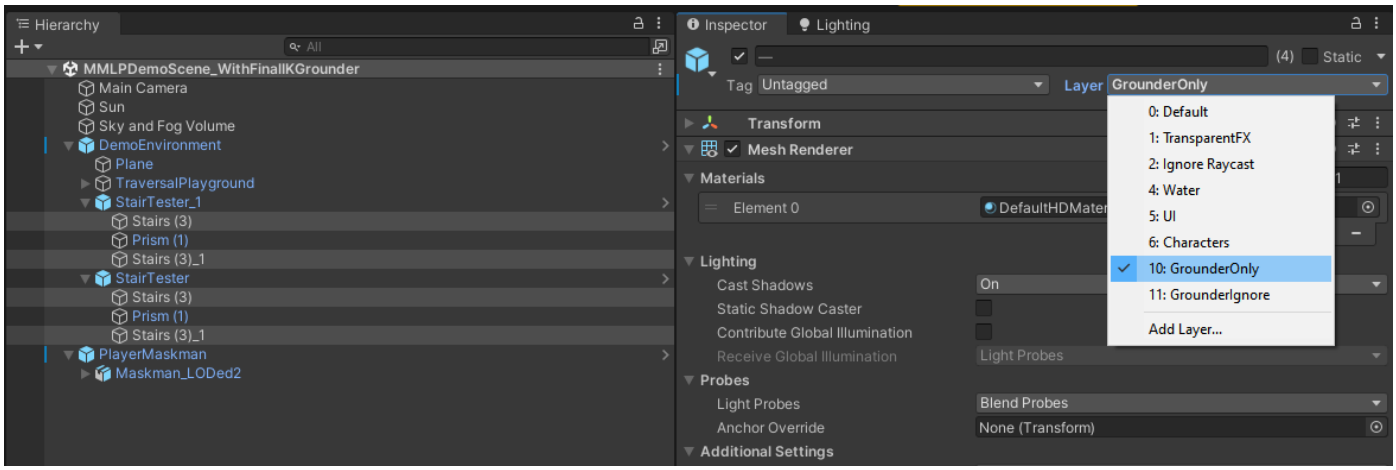
3.6 Using any of the demo scenes that include Final IK

A version of the Core Demo with Final IK grounder integration is in the Demos directory. This may not work directly after import (due to required Layers not being set up in the project), to resolve this:

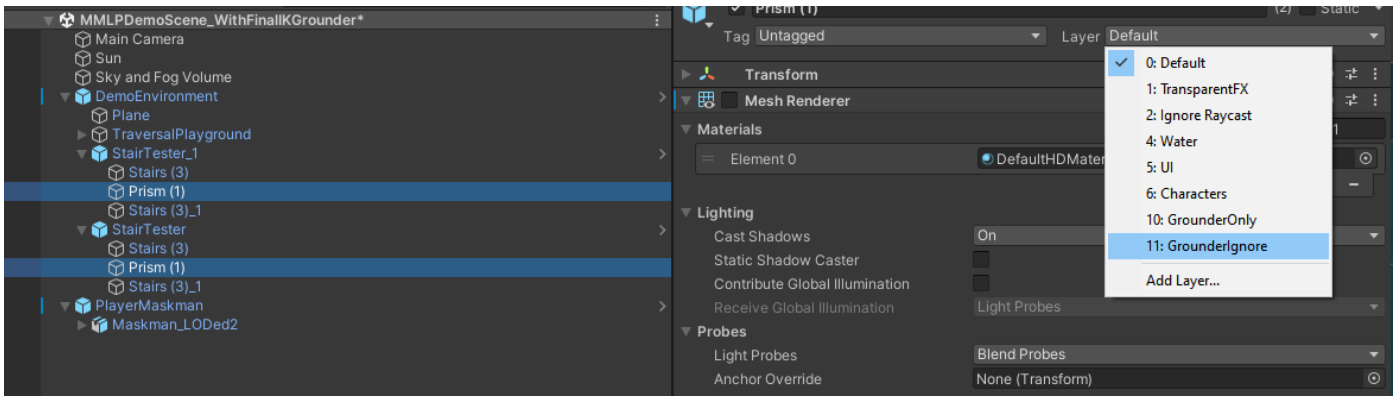
1. Create Layers: GrounderOnly and GrounderIgnore



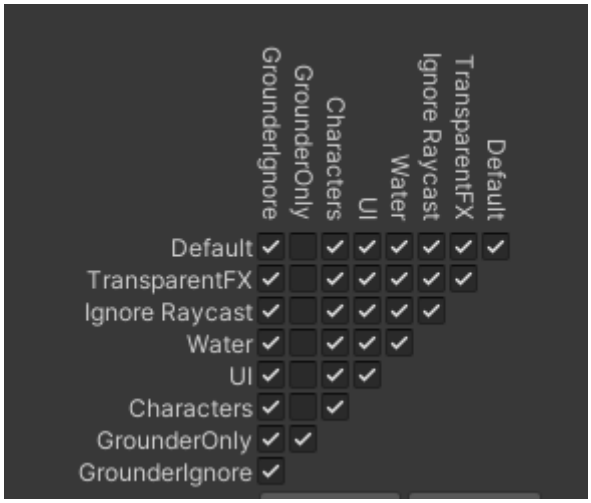
2. Put all stepped-incline objects (stairs) on **Grounder Only** layer



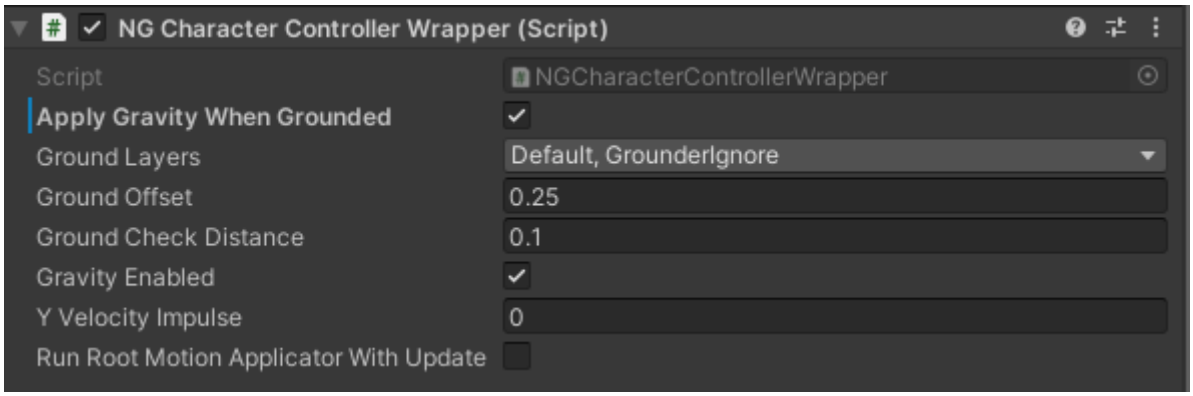
3. Put put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



4. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:



5. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)

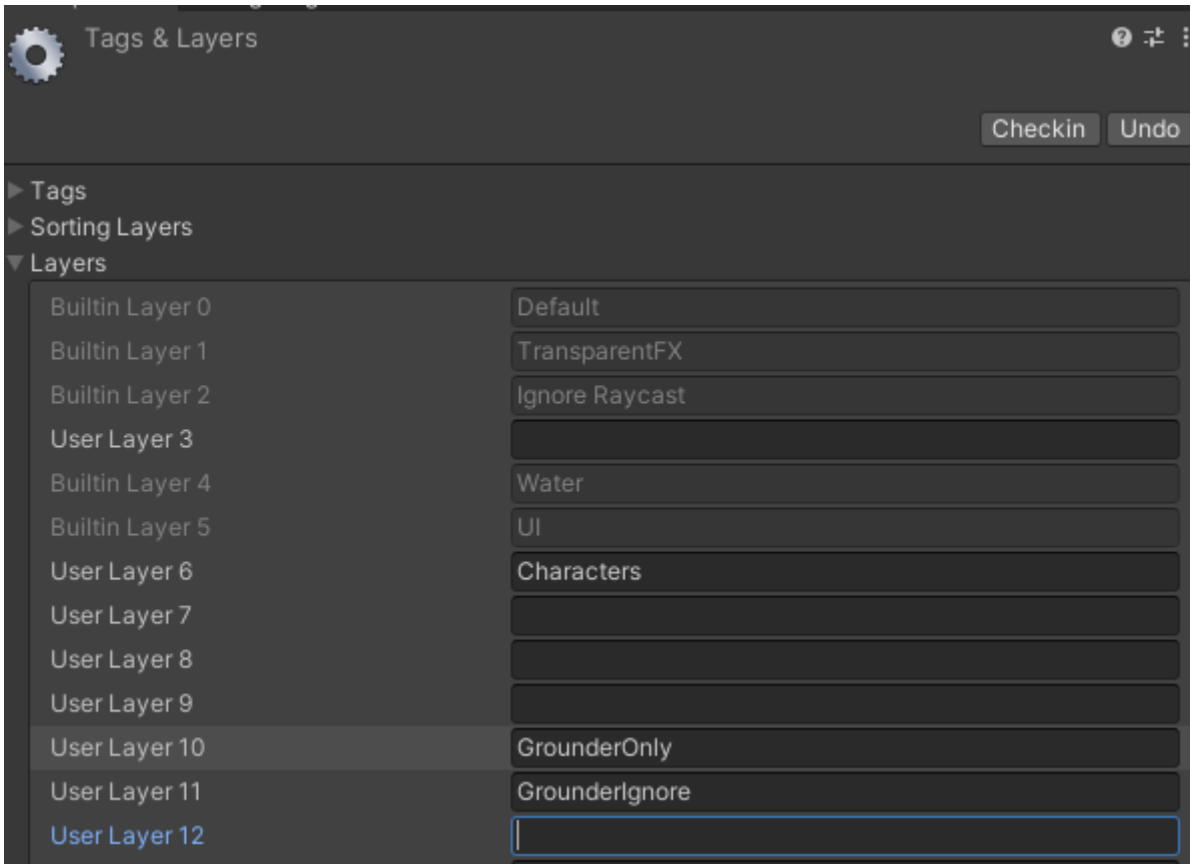


4. Configuration - Final IK Grounder

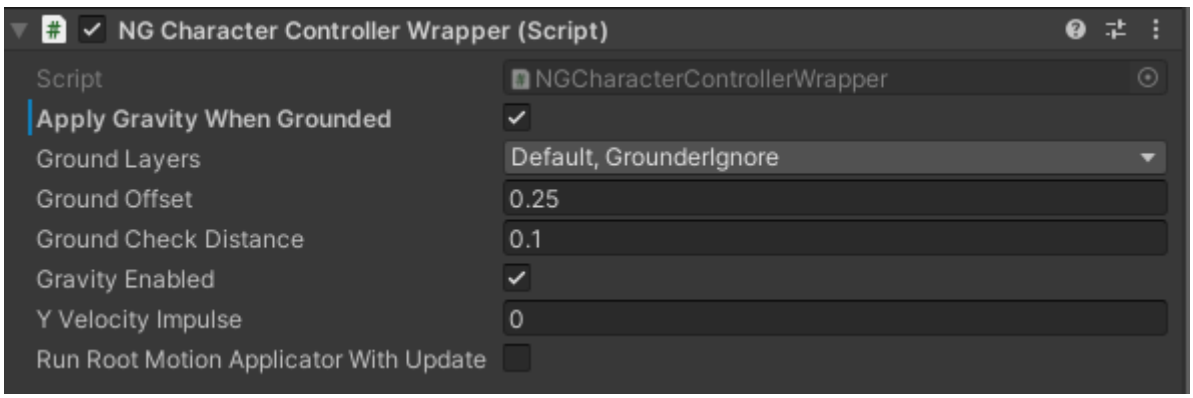
4.1 Setting up your Character and Environment for Final IK grounder

4.1.1 **Environment**

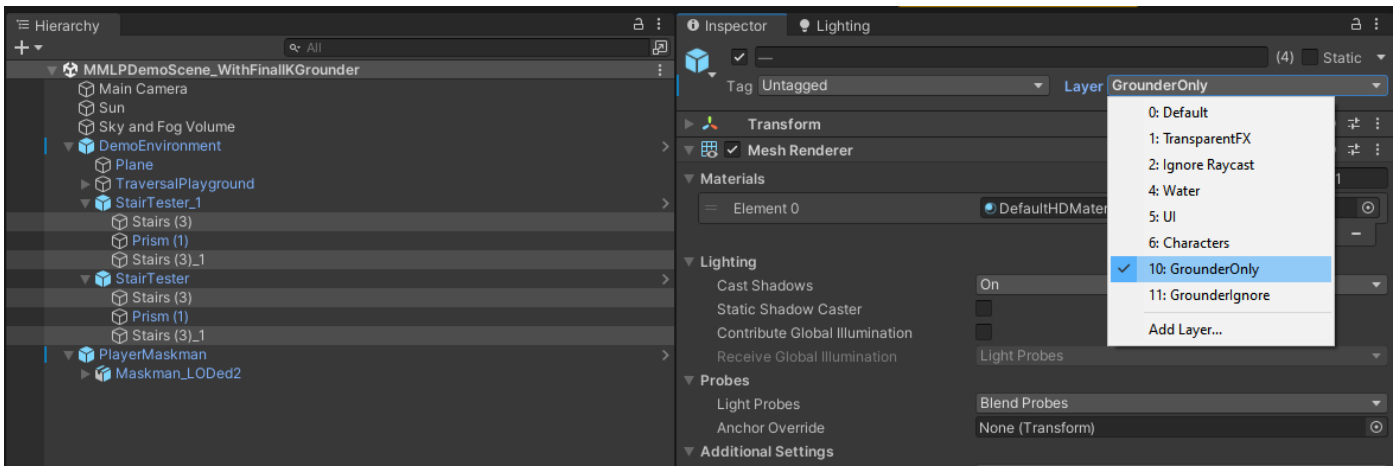
1. Create Layers: GrounderOnly and GrounderIgnore



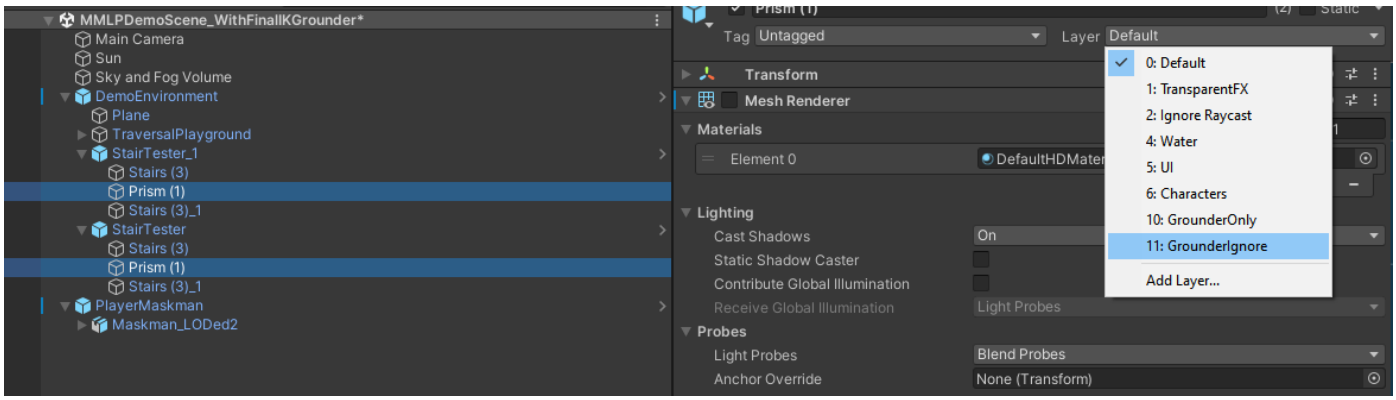
2. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)



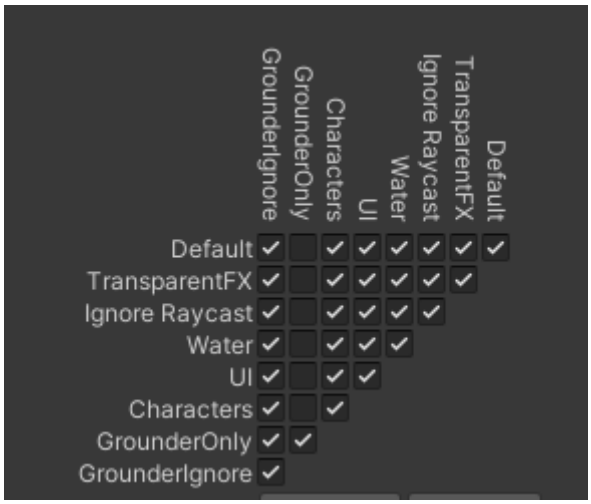
3. Put all stepped-incline objects (stairs) on **Grounder Only** layer



4. Put put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



5. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:

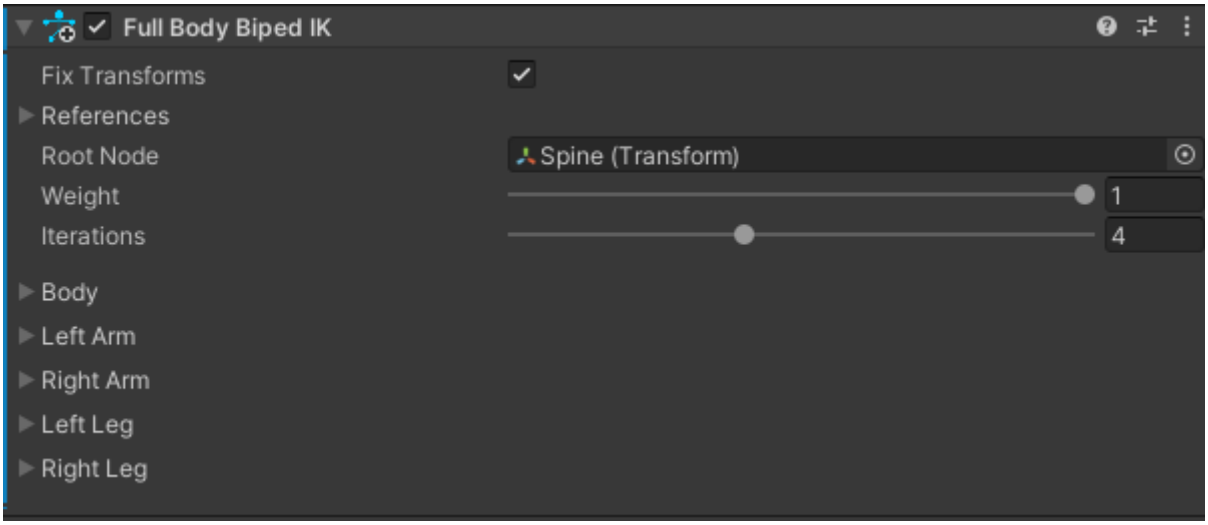


4.1.2 **Character**

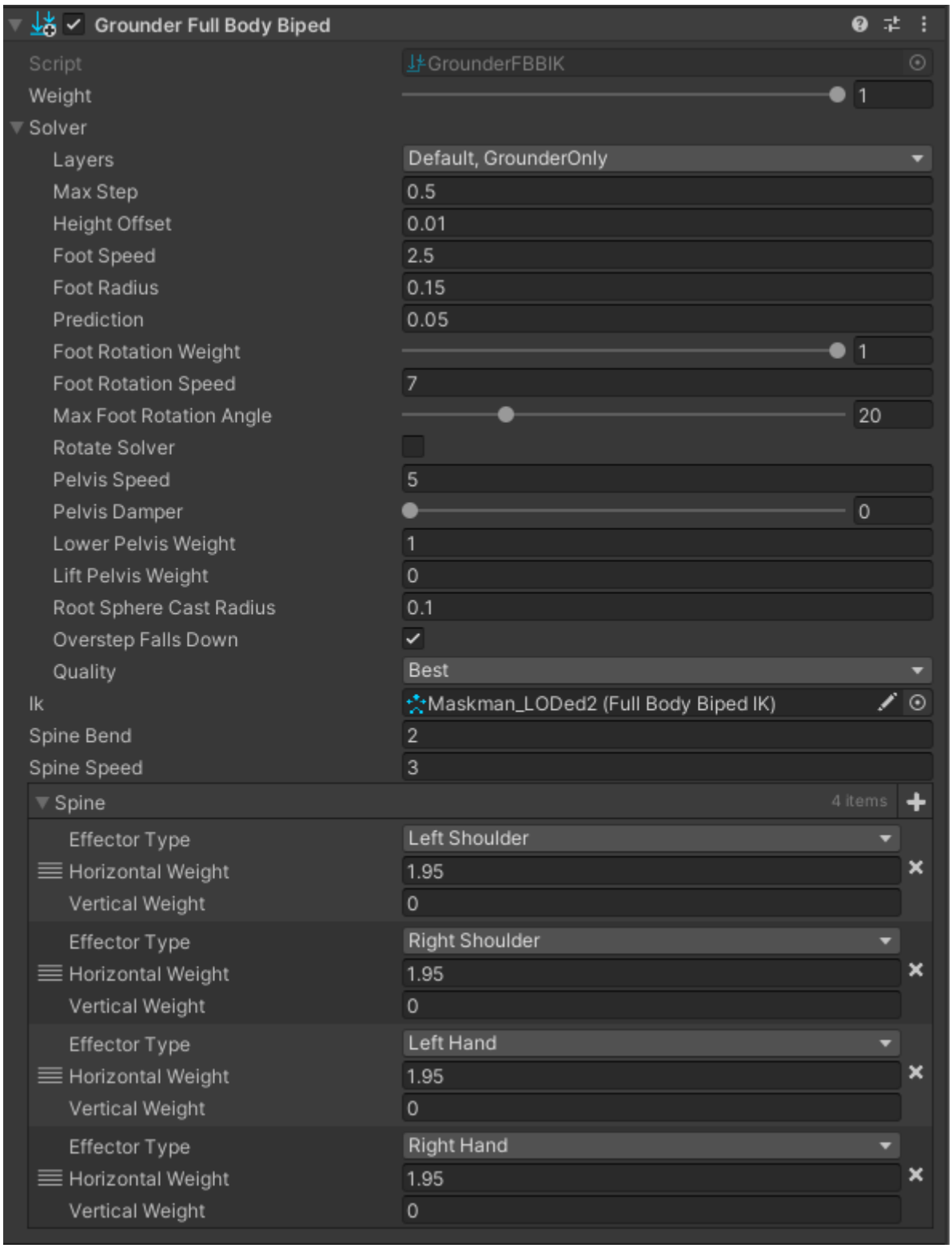
Important

This section is provided for general reference. If you're using our Motion-Matching Locomotion Controller, the config wizard will automatically configure The Biped IK and Grounder components for you. However, if you wish to perform this step manually, or if you don't any Threepeat assets and just want to set up grounder, here you go:

1. To the sub-object of your character that contains the Animator (and MxMAnimator, etc), Add **FBBIK** Component:



2. To that same object, add **Grounder Full Body Biped** component and set settings per the picture below:



a. Make sure All ground layers (except Grounder Ignore) are set in Solver -> Layers

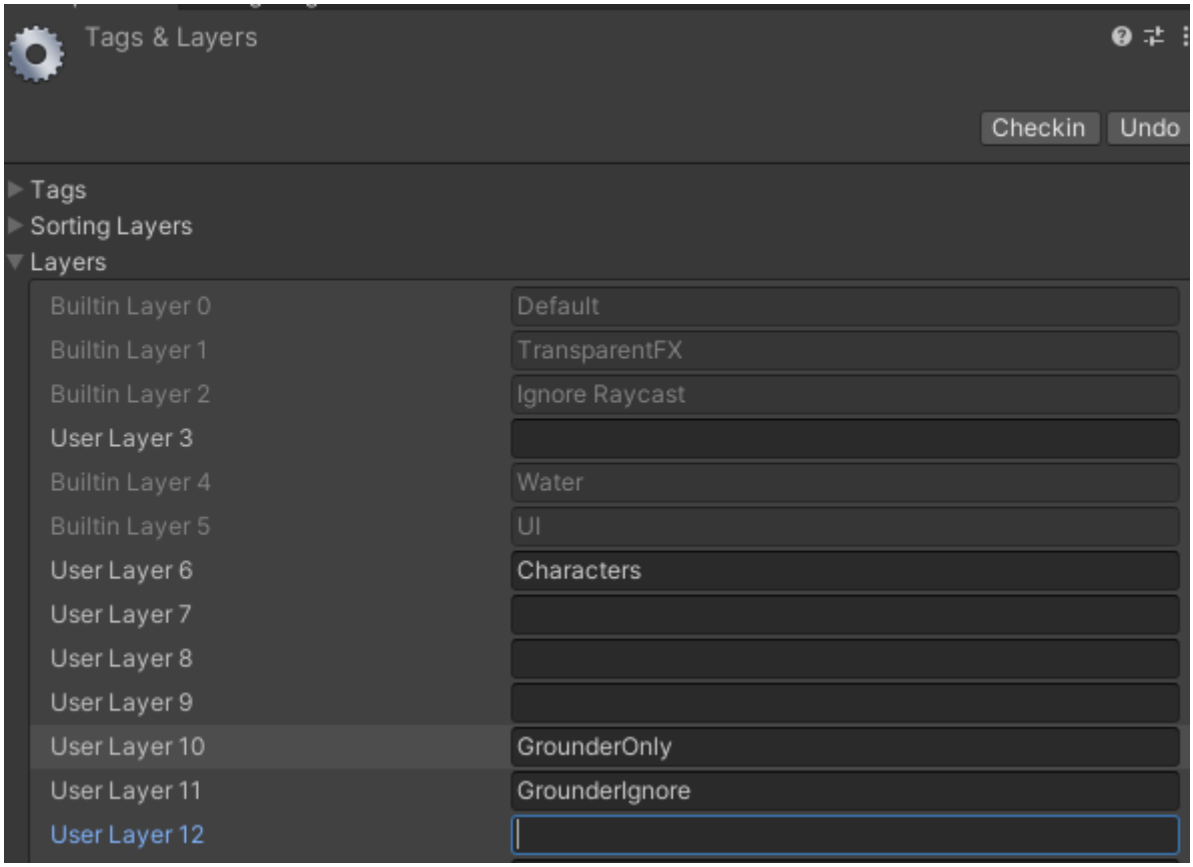
b. Add the Left/Right Shoulders and Left/Right Hands to the Spine list (this causes the player to lean forward when running up inclines and stay more upright when descending inclines to add realism)

3. You're done!

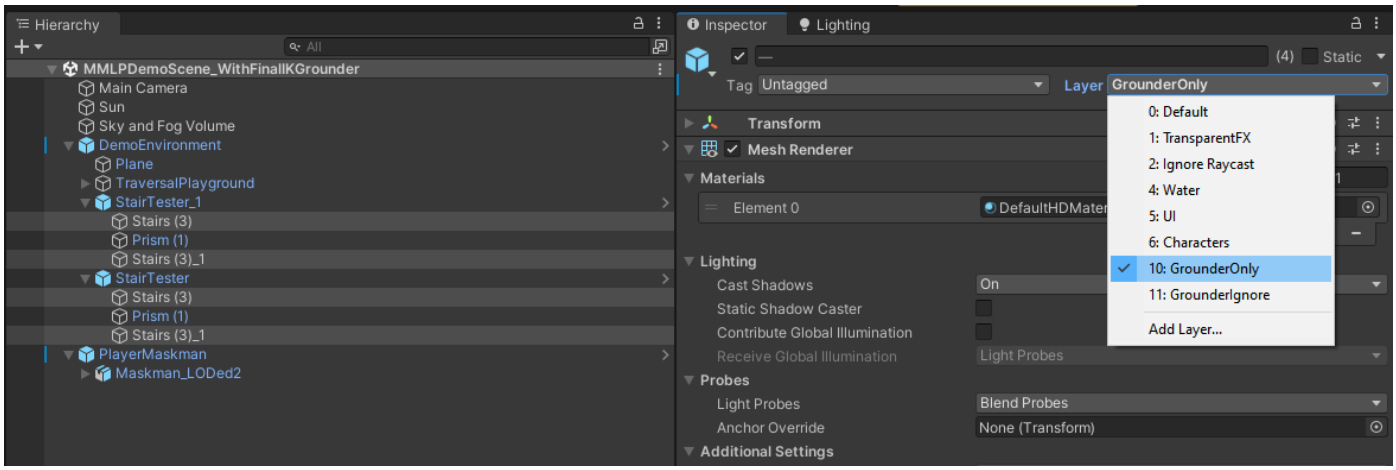
4.2 Using any of the demo scenes that include Final IK

A version of the Core Demo with Final IK grounder integration is in the Demos directory. This may not work directly after import (due to required Layers not being set up in the project), to resolve this:

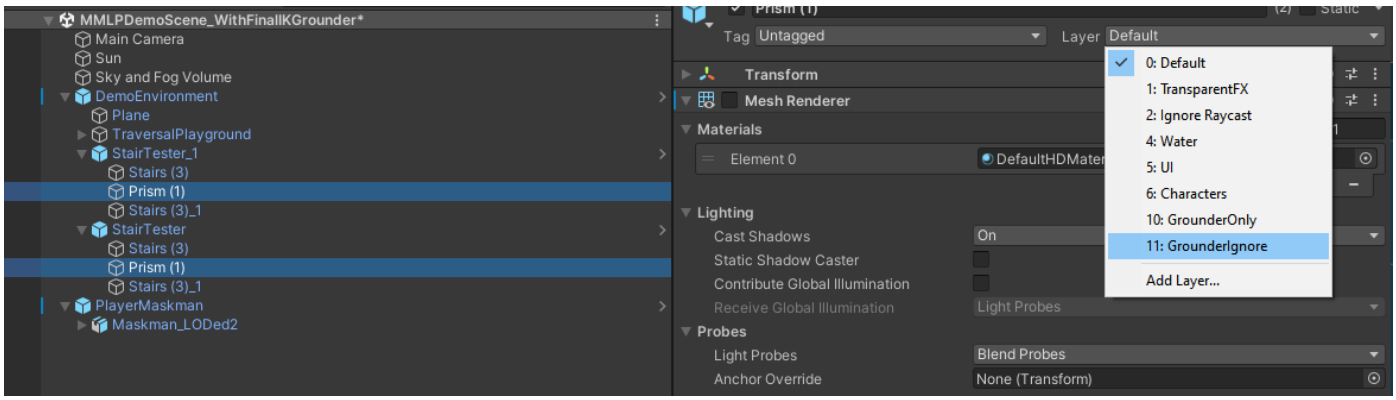
1. Create Layers: GrounderOnly and GrounderIgnore



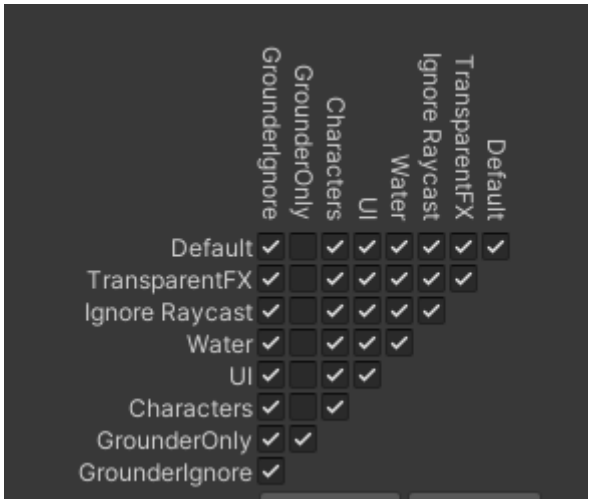
2. Put all stepped-incline objects (stairs) on **Grounder Only** layer



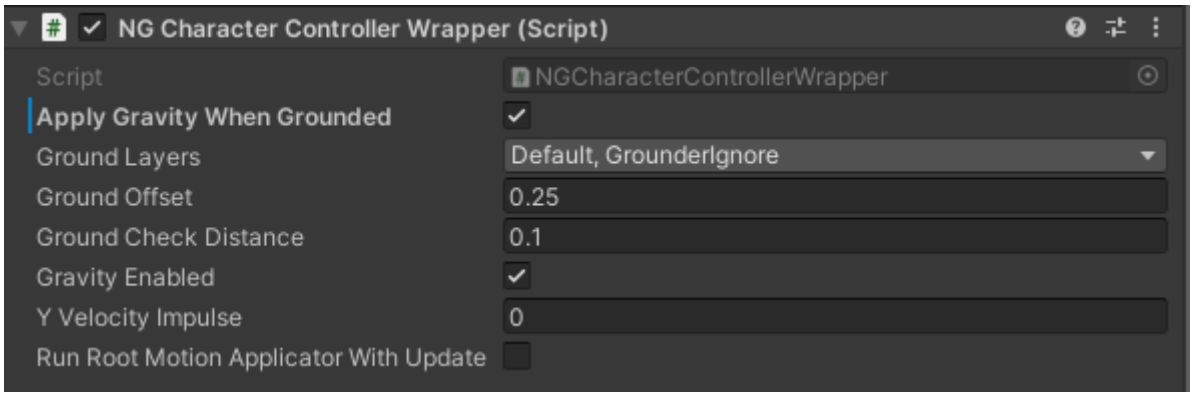
3. Put put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



4. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:



5. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)



5. Demo Scenes and Examples

The MMLP unitypackage contains an example scene (**Assets\Plugins\Threepeat\MMLocoParkour\Demos**) with a fully-configured Humanoid-rigged character to demonstrate all of the functions:

Important

The FinalIKGrounder demo scene will only work if you've setup your project and that scene as described in the **Using MMLPDemoScene_With_FinalIKGrounder Demo Scene** section of Configuration - Final IK Grounder.

Behavior	Action (if needed)
Move camera	mouse
zoom camera	mouse wheel
Running	Arrow keys
smooth walk/run transitions	(Requires Strider for maximum smoothness) Use Gamepad, joystick, or any input axis with graduated [-1,1] values
Sprint	Left Shift (Hold)
Crouch	Left Ctrl
Crouch walk	Arrow keys or joystick while crouching
Jump	Spacebar
Big Jump	Hold Spacebar for > 0.35 seconds or press spacebar while sprinting
Fall detection	Run off of an elevated platform, player will detect falling and switch to Falling Blend-Space based on forward speed
Normal/Hard/ Rolling Landing	Based on impact speed with the ground when landing, Player will immediately resume whatever behavior they were doing prior to the jump or falling condition (Normal), execute a hard landing event with a forward roll (if the path ahead of them is clear), or execute a hard landing event in place (if path ahead of them is blocked)

Note

If you arrived at this page from Installation Step 3, you can continue with configuring your own custom player character at Configuration - Character

5.1 Completed Examples

- Blending between MMLC and the animator (including with Avatar Mask to simultaneously use MMLC and the Animator): See **Example_ToggleAnimatorOnKey**
- Toggle whether character is able to run or only walk: See **Example_ForceWalkOnKey**
- Programmatic Jump triggering: See **Example_MakeCharacterJumpWhenYouPressGKey**
- Change a NavMesh-driven character's destination: See **Example_SetNavMeshTargetOnLKey**
- Switch between NavMesh and InputSystem or InputManager at runtime: See **Example_SwitchInputSchemeToInputSystemOnKey**, **Example_SwitchInputSchemeToNavMeshOnKey**, and **Example_SwitchInputSchemeToInputManagerOnKey**

5.2 Completed Tutorials

All tutorials are available on our Youtube: https://www.youtube.com/@threepeatgames7837/videos

5.3 Scripts from Youtube Tutorials

Working with Mecanim Part 1, Example_ToggleAnimatorTheNewWay.cs